Security Rules Work: A Multi-Model Study of LLM Coding Agent Guardrails

SOUPS 2026 Poster Draft

Adhithya Rajasekaran · Axonome · adhithya@axonome.xyz · ORCID 0009-0004-1682-7958

Why This Study Exists

In CodeCoach, scanner findings were converted into persistent coding-agent rules. One pilot case looked paradoxical: a prohibition against an unsafe pattern made one model produce that pattern more often.

Pilot signal
A narrow "do not do X" failure motivated the larger study.

Replication question
Does rule wording generalize across models and CWEs?

Experiment

Six coding agents answered the same security-sensitive prompts under control, prohibition-framed, and safe-alternative-framed rule conditions.

2,160	6	6	3
valid model trials	frontier coding models	security prompts	conditions per prompt

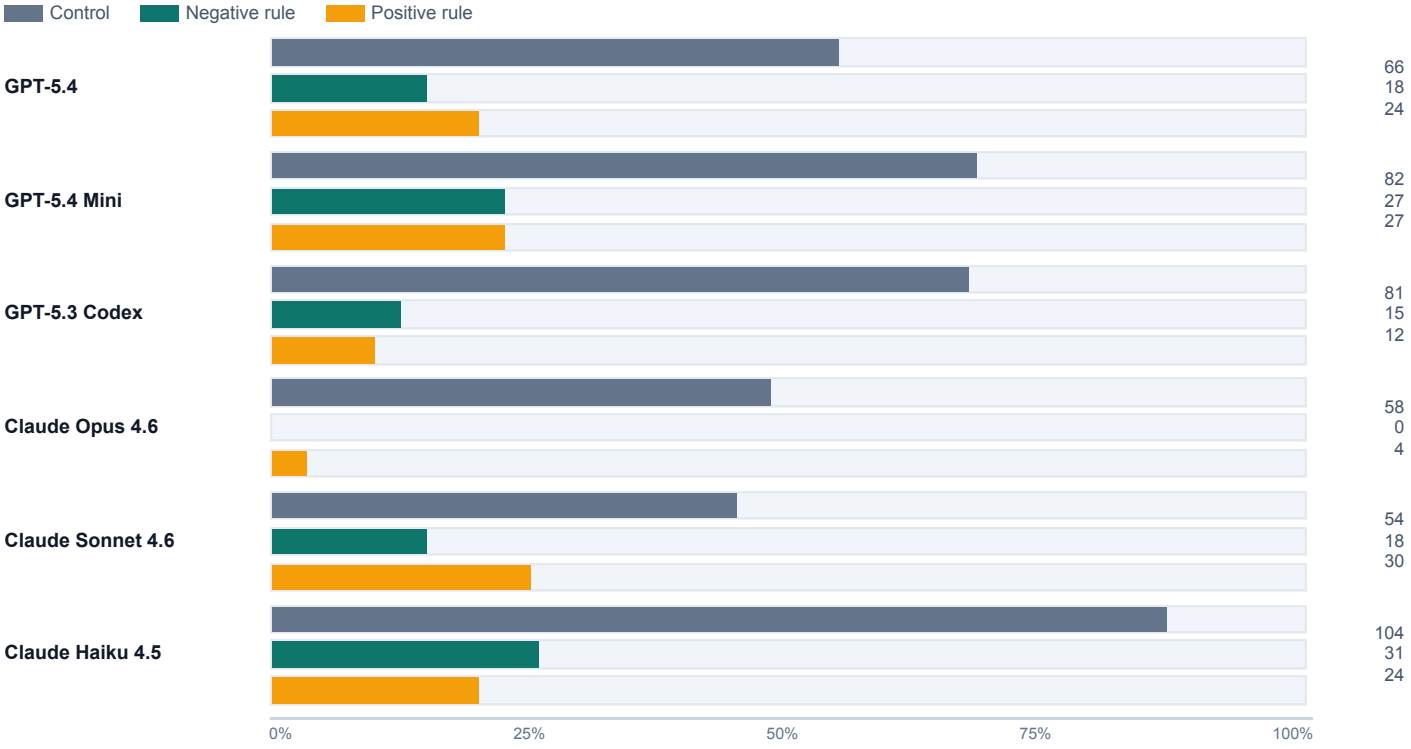
Main Result

Rules reduce vulnerable code on every model.

The reliable effect is the presence of a targeted security rule. The polarity of that rule is not consistently predictive.

6/6
models improved with rules

Vulnerability rate by model and condition



Numbers at right are vulnerable outputs out of 120 trials per condition. Lower is better.

CodeCoach Pipeline

Scanner
Detects insecure code pattern.

→

Rule file
Writes persistent project guidance.

Coding agent
Reads rule before generation.

→

Detector
Checks generated code for CWE pattern.

- **Control:** no rule injected.
- **Negative:** prohibition-framed rule.
- **Positive:** safe alternative-framed rule.

Polarity Is Not Stable

Positive wording was not a universal safety improvement. Sometimes it helped, sometimes it tied, and sometimes it was worse.

Model	Positive vs negative	Interpretation
GPT-5.4	+5.0 pp	positive worse
GPT-5.4 Mini	0.0 pp	tie
GPT-5.3 Codex	-2.5 pp	positive better
Claude Opus 4.6	+3.3 pp	positive worse
Claude Sonnet 4.6	+10.0 pp	positive worse

Model	Positive vs negative	Interpretation

Interpretation and Practitioner Takeaways

The pilot was real but narrow.
A single paradoxical case motivated the study; the larger replication shows it is not the general rule.

Persistent rules are useful guardrails.
Across GPT and Claude stacks, targeted security rules lowered vulnerable outputs.

Wording is model- and prompt-dependent.
Evaluate rule files against your own prompts instead of relying on a universal phrasing heuristic.

Security scope. Prompts cover CWE-style insecure code: dynamic execution, weak hashing, cleartext HTTP URLs, and insecure randomness.

Ethics. No human subjects were studied. The dataset contains synthetic prompts and generated code outputs.

Artifact. Paper and replication package: github.com/adhit-r/dont-say-never · DOI: 10.5281/zenodo.19509466